AlphaTransit: Learning to Design City-scale Transit Routes

Bibek Poudel [1] **Sai Swaminathan** [1] **Weizi Li** [2]

Abstract

In sequential design problems with delayed rewards, an agent assembles a solution step by step before the quality of each choice is revealed at terminal evaluation. A deceptive optimization landscape can compound the difficulty of this task, as locally attractive choices may yield globally poor structure. Transit network design exemplifies both challenges: greedily serving high-demand corridors can create bottlenecks degrading global performance. We introduce AlphaTransit, a planning-and-learning framework integrating Monte Carlo Tree Search with a graph neural policy-value network. By evaluating downstream consequences before committing, search-guided learning escapes locally attractive designs that trap end-to-end methods. For realistic evaluation, we contribute a Bloomington, Indiana benchmark with topologically accurate roads and census-derived demand. Across demand levels, AlphaTransit consistently outperforms heuristic, evolutionary, and End-to-End Reinforcement Learning (RL) baselines. Under high demand, AlphaTransit serves 85.5% of passengers with 5.95-minute waits versus 66.1% and 13.42 minutes for End-to-End RL. Relative to evolutionary baselines, this yields 35% shorter waits with 17% fewer buses. These gains suggest search-guided learning navigates deceptive landscapes more effectively than gradient-based or evolutionary methods.

1. Introduction

Across combinatorial optimization, a recurring challenge appears in sequential design problems: an agent must assemble a solution step by step, yet the quality of most choices is only revealed at the end. The *Transit Route Network Design Problem* (TRNDP) is a representative real-world instance. It involves selecting a set of possibly overlapping transit routes embedded in a shared street network, determining stop sequences, and setting service frequencies to manage many-to-many passenger flows while respecting operational constraints such as fleet size and cost (Schmidt & Schöbel, 2024; Fan, 2004). These requirements distinguish TRNDP from classical routing problems such as the Traveling Salesman Problem, which optimizes a single tour visiting each node exactly once, and the Vehicle Routing Problem, which assigns capacity-constrained vehicles to serve disjoint customer sets (Ceder, 2016).

The complexity of TRNDP arises from four interacting properties that together stress the limits of gradient based methods. First, the feasible solution space is combinatorial and the problem is NP-hard, with the number of valid configurations growing exponentially in network size and route length (Kepaptsoglou & Karlaftis, 2009; Owais, 2026). In the Bloomington instance studied in this work, designing 16 routes of up to length 14 yields a search space on the order of 9.5×10^{115} configurations (Appendix C), far beyond exhaustive search or systematic local improvement (Newell, 1979). Second, the optimization landscape can be deceptive: locally attractive choices that serve high-demand corridors early can create transfer bottlenecks, correlated delays, and redundant service that reduces global throughput. Third, rewards are delayed and strongly nonlocal: small changes to one route can reroute passenger flows and shift loads across distant segments, yet these effects are only measurable after the full design is evaluated end-to-end in simulation. Fourth, the objective is inherently multi-criteria, with passenger-centric goals (coverage, service rate, low waiting and travel time) competing with operator-centric constraints (fleet size, overlap, and utilization).

Existing approaches to TRNDP span two paradigms. Classical methods rely on metaheuristics, including genetic algorithms (Mumford, 2013), simulated annealing (Zhao & Zeng, 2006), and bee colony optimization (Nikolić & Teodorović, 2013), that can explore broadly but require substantial domain expertise and can be brittle across instances (Durán-Micco & Vansteenwegen, 2022). Recently, RL has been explored either to learn route construction policies directly (Yoo et al., 2023) or to guide metaheuristic search (Holliday et al., 2025). However, pure policy learning can be sample-inefficient in sparse-reward settings (Sutton

[1]Min H. Kao Department of Electrical Engineering and Computer Science, University of Tennessee, Knoxville, TN, USA [2]Department of Computer Science and Engineering, University of California, Riverside, CA, USA. Correspondence to: Bibek Poudel <bpoudel3@vols.utk.edu>.

Preprint.

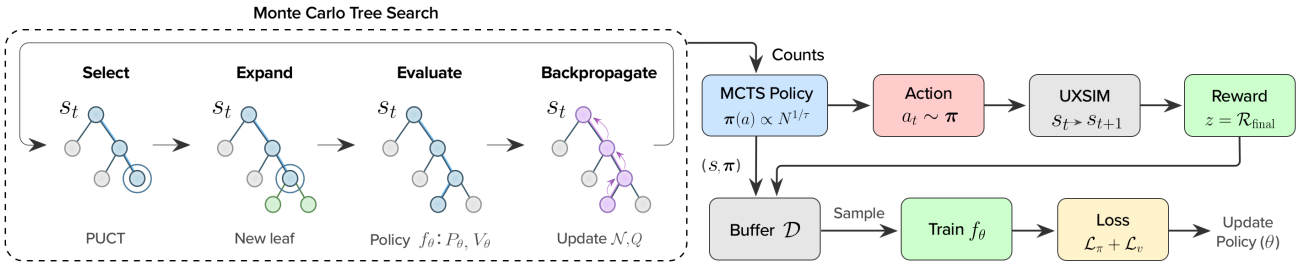


Figure 1. **AlphaTransit overview.** At each route design step, Monte Carlo Tree Search (MCTS) repeatedly performs Selection, Expansion, Evaluation, and Backpropagation using Predictor Upper Confidence bounds on Trees (PUCT), guided by the policy network f_θ. After many simulations, the visit-count policy π selects the next action, which is applied to the mesoscopic traffic simulator (UXSim) to advance the design state. Once the full route set is constructed, the simulator returns a terminal reward z. Visited states, their search-improved policies, and reward z are stored in a replay buffer $\mathcal{D}$ and are sampled to update f_θ by jointly fitting policy and value targets.

& Barto, 2018), and most RL-based methods have been evaluated only on small synthetic networks that fail to capture the topological structure and demand patterns of real cities (Heyken Soares et al., 2019; Vermeir et al., 2021).

A complementary strategy is to incorporate explicit lookahead. In sequential decision problems with sparse terminal rewards, integrating learned models with Monte Carlo Tree Search (MCTS) provides a strong form of policy improvement: search evaluates downstream consequences before committing, and the resulting search statistics provide richer training signal than trajectory returns alone (Silver et al., 2018; Schrittwieser et al., 2020). This template is well matched to TRNDP, where locally appealing route extensions can lock the design into poor transfer structure and the reward signal arrives only after completing a full network. Accordingly, we introduce AlphaTransit, a planning and learning framework for transit network construction that combines a graph attention policy-value network with MCTS (Figure 1). At each construction step, the policy-value network proposes candidate route extensions and estimates eventual network quality, while MCTS performs lookahead search to refine these proposals. Because training uses only terminal simulation outcomes, AlphaTransit optimizes directly for system-level performance without requiring dense shaping rewards. Our contributions are:

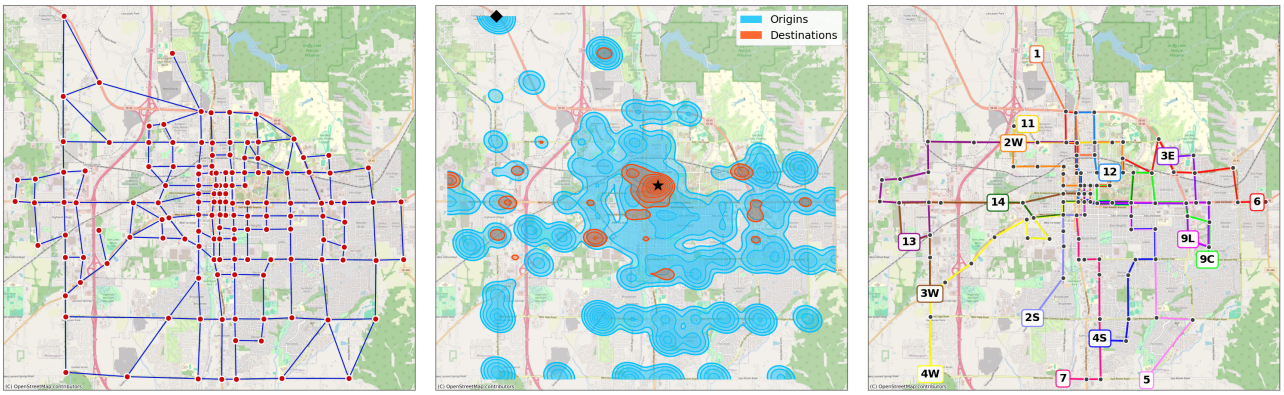
- We propose AlphaTransit, a search-guided learning framework for sequential transit route construction that combines MCTS with a learned graph policy-value network trained from terminal simulation outcomes.
- We introduce a realistic TRNDP benchmark based on Bloomington, Indiana, with a topologically accurate road graph, census-derived OD demand, and the existing human-designed network as a real-world reference.
- We provide systematic experiments across two modal-split scenarios and multiple baselines, demonstrating consistent improvements in passenger-facing performance and competitive operator efficiency.

We evaluate AlphaTransit on the Bloomington benchmark under two modal-split scenarios. Under typical urban demand ($\alpha = 0.3$), AlphaTransit achieves a service rate of 61.34% with an average wait time of 6.90 minutes, outperforming End-to-End RL and heuristic baselines while attaining the highest route efficiency. Under high transit demand ($\alpha = 1.0$), AlphaTransit serves 85.50% of passengers with an average wait time of 5.95 minutes and an average travel time of 35.08 minutes, representing a 35% wait-time reduction relative to the Genetic Algorithm while deploying 17% fewer buses (167 versus 202). Qualitatively, the learned networks exhibit distinct structure: rather than maximizing geographic coverage, search-guided learning concentrates service on high-throughput corridors, achieving the highest service rates with fewer route-kilometers. This concentration emerges from lookahead search exposing downstream bottlenecks invisible to greedy policies.

2. Related Work

Transit route network design and frequency setting has been studied for decades and is typically posed as a multi-objective, NP-hard optimization that couples route layout, passenger assignment, and operating resources (Kepaptsoglou & Karlaftis, 2009; Durán-Micco & Vansteenwegen, 2022; Owais, 2026). Because exact methods are are generally intractable and do not scale to realistic networks, most practical approaches rely on heuristics and metaheuristics, including evolutionary search, simulated annealing, and swarm-based methods (Mumford, 2013; Zhao & Zeng, 2006; Nikolić & Teodorović, 2013). While these methods can produce strong solutions on standard benchmarks (e.g., the Mandl network (Mandl, 1980)), they often require substantial domain expertise of move operators, penalties, and tuning schedules, and their performance can be sensitive to instance-specific structure and constraints (Durán-Micco & Vansteenwegen, 2022; Vermeir et al., 2021).

Motivated by progress in neural combinatorial optimization and graph-based decision making, recent work has explored reinforcement learning (RL) for transit design to construct



(a) (b) (c)

Figure 2. (a) The Bloomington transportation network with 143 nodes and 243 edges. (b) Spatial distribution of trip origins (blue) and destinations (red) showing concentrated activity areas with the highest demand origin of 344 veh/hr from node 1 (⧫), and the highest demand destination of $1,681$ veh/hr to node 129 (★). (c) The 16 transit routes currently operating in the city.

route sets and frequencies directly or to learn heuristics that guide iterative improvement (Bello et al., 2016; Khalil et al., 2017; Yoo et al., 2023; Darwish et al., 2020; Li et al., 2023; Holliday & Dudek, 2023; 2024; Holliday et al., 2025). These approaches reduce reliance on hand-crafted operators, but they inherit two recurring challenges in sequential design: rewards are naturally sparse and delayed because performance is only revealed after a full network is assembled, and credit assignment across many construction steps can be unstable (Sutton & Barto, 2018). In addition, evaluations are frequently performed on small or stylized networks, which can understate the difficulty introduced by irregular topology, heterogeneous demand, and long-range interactions between route choices and passenger flows (Heyken Soares et al., 2019; Yoo et al., 2023).

A complementary strategy for sparse-reward sequential decision problems is to couple learned policy and value networks with explicit lookahead via Monte Carlo Tree Search (MCTS), using search to select actions and produce improved policy targets for training (Silver et al., 2018; Schrittwieser et al., 2020). Within transportation and spatial network settings, MCTS has been applied to related but distinct problems: multi-objective tree search has been used to enumerate Pareto-optimal routes between a single origin and destination pair, relying on heuristic subspace estimation rather than learned priors (Weng et al., 2020); planning-based MCTS has been applied to generic spatial network augmentation, optimizing abstract structural metrics without a learning component or demand-driven evaluation (Darvariu et al., 2023); and deep RL with MCTS has been combined for metro network expansion, though under a different decision space and evaluation pipeline than bus-route TRNDP (Alkilane & Lee, 2025). Broader work highlights adaptations of MCTS for settings beyond two-player games, including neural guidance for single-player problems (Kemmerling et al., 2023) and surrogate-assisted approaches for expensive evaluations (Amiri et al., 2024). Our work instantiates this planning-and-learning template for multi-route transit network construction: we use search-improved action distributions to train a reusable policy and value model, ground learning in stochastic terminal feedback from traffic simulation, and target system-level objectives on a realistic city-scale benchmark without dense shaping rewards.

3. Methodology

3.1. Real-world Network and Demand Data

We propose a new real-world transit design dataset that includes: (i) a topologically correct road graph with 143 nodes and 243 bidirectional edges, extracted from the street network in Bloomington, Indiana; (ii) a block-level OD demand matrix extracted from U.S. Census data; and (iii) the 16 existing Bloomington Transit routes (Bloomington Transit). Unlike many benchmarks that rely on synthetic networks and demand, our dataset provides greater realism in the design and evaluation of transit networks. Figure 2 visualizes the network, the spatial distribution of origins and destinations, and the current routes. For processing details and assumptions, see the supplementary material section B.

3.2. Markov Decision Process

We formulate the TRNDP as a finite-horizon Markov decision process defined by the tuple $(\mathcal{S}, \mathcal{A}, P, \mathcal{R})$ on a city graph $G = (V, E)$ with $n = |V|$ nodes and an OD matrix $D \in \mathbb{R}_{\geq 0}^{n \times n}$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ is the deterministic transition function, and $\mathcal{R} : \mathcal{S} \to \mathbb{R}$ is the reward obtained by evaluating completed route sets via traffic simulation. In each episode, the agent sequentially designs K routes $\Pi = \{r_1, \ldots, r_K\}$, each with a maximum length of $L_{\max}$ nodes, by extending one route at a time until all routes are finalized. We seek to learn a policy that balances passenger and operator objectives: maximizing network coverage and service rate while minimizing

travel time (passenger-side), and penalizing excessive route overlaps while encouraging high utilization (operator-side). The learning objective is to maximize expected reward:

$$\max_{\theta} \quad J(\theta) \;=\; \mathbb{E}_{\pi_\theta}\big[\mathcal{R}(\Pi)\big], \tag{1}$$

where π_θ is the policy parameterized by θ and Π is the complete route set constructed under that policy. The policy operates under the following constraints, which we formalize in Appendix A, including definitions of the road network (Definition 1), transit routes and networks (Definitions 2–3), sequential route construction (Definition 6), and the Transit Route Network Design Problem (Definition 8):

- *Fixed route count*: The designed solution comprises exactly K routes, i.e., $|\Pi| = K$.
- *Graph connectivity*: Each route r_k forms a simple path on G, where consecutive nodes must share an edge; formally, for any adjacent pair (v_i, v_{i+1}) in r_k, we require $\{v_i, v_{i+1}\} \in E$.
- *Bidirectional operation*: Each route $r_k = (v_1, v_2, \ldots, v_\ell)$ is operated in both directions, with buses traversing the forward sequence $v_1 \to v_2 \to \cdots \to v_\ell$ and the reverse sequence $v_\ell \to v_{\ell-1} \to \cdots \to v_1$, enabling bidirectional O-D connectivity along the route.
- *Simple paths*: Routes are self-avoiding walks with no node appearing more than once within a single route.
- *Length bounds*: Each route satisfies $L_{\min} \le |r_k| \le L_{\max}$, where $|r_k|$ is the number of nodes in route r_k. In our setting, $K = 16$ and $L_{\max} = 14$, yielding an episode horizon of $T = K \cdot L_{\max} = 224$ steps.
- *Sequential construction*: Routes are built by extending one node at a time from a frontier node (the most recently added node). At each step, the action space is restricted to the one-hop neighbors of the frontier that have not yet been visited in the current route being constructed. A route terminates upon reaching $L_{\max}$ or when no valid neighbors remain.

State. At each decision step t, the design state encodes (i) the static road graph structure and edge attributes, and (ii) the partial transit network constructed so far. We represent the state as $s_t = (X_t, \mathcal{I}, Z)$ where $\mathcal{I}$ is a directed edge list for message passing, Z stores edge features (e.g., length and free-flow speed), and $X_t \in \mathbb{R}^{n\times d}$, where d is the node feature dimension, contains node features capturing geometry, OD demand aggregates, and indicators for membership in the current route and previously completed routes. The full state specification is provided in Appendix F.

Action. At each step, the agent selects an action $a_t \in \mathcal{A} = \{1, \ldots, n\}$ to append a node to the current route. Since the admissible candidate set $\mathcal{C}_t$ (one-hop neighbors of the frontier not yet in the current route) satisfies $|\mathcal{C}_t| \ll n$ and this gap widens with network size, we use action masking to concentrate probability mass on valid choices (Huang & Ontañón, 2020). We form a binary mask $m_t \in \{0,1\}^n$ over $\mathcal{C}_t$ and the policy emits logits $\ell \in \mathbb{R}^n$, sampling from a masked categorical distribution:

$$\pi_\theta(a = i \mid s_t) \;=\; \begin{cases} \dfrac{\exp(\ell_i)}{\sum_{j:\, m_t[j]=1} \exp(\ell_j)} & \text{if } m_t[i] = 1, \\ 0 & \text{if } m_t[i] = 0. \end{cases} \tag{2}$$

This places zero probability on nodes outside $\mathcal{C}_t$. Node feasibility is exposed in the state via valid-next flag in X_t, informing the policy and allowing it to condition on action constraints during training. When $\mathcal{C}_t = \varnothing$, the current route terminates and construction of the next route begins.

Reward. After all K routes are constructed, we deterministically assign each route a frequency of service using a max-load principle (Appendix D), then evaluate the resulting network via simulated passenger flows. The reward is a weighted combination of system-level passenger and operator metrics, prioritizing passenger efficiency through coverage, service rate, and travel time, while controlling operator-side costs via overlap, fleet size, and utilization:

$$\begin{aligned} \mathcal{R} = b_0 \cdot \Psi + b_1 \cdot \sigma - b_2 \cdot \bar{t}_{\text{wait}} - b_3 \cdot \bar{t}_{\text{move}} \\ - b_4 \cdot \omega - b_5 \cdot \frac{F}{K} + b_6 \cdot u, \end{aligned} \tag{3}$$

with $b_0 = 60$, $b_1 = 45$, $b_2 = 20$, $b_3 = 10$, $b_4 = 10$, $b_5 = 2$, and $b_6 = 12$, chosen through Bayesian hyper-parameter tuning. Here Ψ is the demand coverage potential

$$\Psi = \frac{\sum_{(i,j)\in\mathcal{P}_{\text{reachable}}} D_{ij}}{\sum_{i,j\in V} D_{ij}}, \tag{4}$$

where D_{ij} denotes passenger flow from node i to node j, and $\mathcal{P}_{\text{reachable}} = \{(i,j) : \exists \text{ a path from } i \text{ to } j \text{ in } \Pi\}$. The service rate $\sigma = N_{\text{boarded}}/N_{\text{want}}$ measures the fraction of potential riders who boarded a bus, where N_{boarded} is the number of passengers who boarded buses during the simulation and N_{want} denotes the total number of potential riders. The remaining terms are average wait time $\bar{t}_{\text{wait}}$, average in-vehicle time $\bar{t}_{\text{move}}$, route overlap ratio ω, fleet size F, and bus utilization u. The reward is normalized online (Huang et al., 2022; Coholich, 2023).

3.3. AlphaTransit

AlphaTransit is a planning-based approach to transit route design that combines Monte Carlo Tree Search (MCTS) with neural network guidance (Silver et al., 2018). At each decision step, it performs explicit lookahead via MCTS

Algorithm 1 AlphaTransit

Input: Graph G, OD matrix D, edges $\mathcal{I}$, features Z, routes K, max length $L_{\max}$, MCTS iters N_{iter}.
Output: Policy f_θ (yields routes $\Pi = \{r_1, \ldots, r_K\}$)
Initialize: Buffer $\mathcal{D} \leftarrow \varnothing$
while $t < t_{\max}$ **do**
  // Parallelized search-generated episodes
  for each worker $w \in \{1, \ldots, W\}$ **in parallel do**
    $\Pi \leftarrow \varnothing$; $V_{\text{cmp}} \leftarrow \varnothing$; $E \leftarrow [\,]$
    for $k = 1$ to K **do**
      $r_k \leftarrow [\,]$; $V_{\text{cur}} \leftarrow \varnothing$
      while $|r_k| < L_{\max}$ **do**
        $\mathcal{C}_t \leftarrow$ valid one-hop neighbors of frontier
        if $\mathcal{C}_t = \varnothing$ **then**
          break
        end if
        $s_t \leftarrow \textsc{FormState}(V_{\text{cur}}, V_{\text{cmp}}, \mathcal{I}, Z)$
        for $i = 1$ to N_{iter} **do**
          Select: leaf node via PUCT (Eq. 5)
          Expand: if leaf unexpanded, add children
          Evaluate: $(P, V) \leftarrow f_\theta(s)$
          Backprop: $N_{s,a} \mathrel{+}= 1$; $W_{s,a} \mathrel{+}= V$
        end for
        $\boldsymbol{\pi}_t(a) \propto N(s_t, a)^{1/\tau}$
        $E.\textsc{Append}((s_t, \boldsymbol{\pi}_t))$
        $a_t \sim \boldsymbol{\pi}_t$; append a_t to r_k
        $V_{\text{cur}} \leftarrow$ nodes in r_k
      end while
      $V_{\text{cmp}} \leftarrow V_{\text{cmp}} \cup V_{\text{cur}}$; $\Pi \leftarrow \Pi \cup \{r_k\}$
    end for
    $z_w \leftarrow \mathcal{R}_{\text{final}}(\Pi)$
  end for
  // Aggregate and add to buffer
  for $w = 1$ to W **do**
    $\tilde{z}_w \leftarrow \textsc{Normalize}(z_w)$; add $(s_i, \boldsymbol{\pi}_i, \tilde{z}_w)$ to $\mathcal{D}$ for all $(s_i, \boldsymbol{\pi}_i) \in E_w$
  end for
  // Policy network optimization
  for step $= 1$ to N_{steps} **do**
    for minibatch $(s, \boldsymbol{\pi}, \tilde{z}) \subseteq \textsc{Sample}(\mathcal{D})$ **do**
      $(P_\theta, V_\theta) \leftarrow f_\theta(s)$; mask + renormalize P_θ
      $\theta \leftarrow \theta - \eta \nabla_\theta \big(-\boldsymbol{\pi} \cdot \log P_\theta + (\tilde{z} - V_\theta)^2\big)$
    end for
  end for
  Broadcast θ to workers
end while
return f_θ

and distills the resulting improved action distribution into a policy-value network. Algorithm 1 presents the complete procedure, where V_{cur} denotes nodes in the current route and V_{cmp} denotes nodes across all completed routes. Training runs for $t_{\max}$ environment steps across W parallel workers using learning rate η; Algorithm supporting functions are defined in Appendix A.

We employ a neural network $f_\theta(s) = (P_\theta, V_\theta)$ that outputs a policy prior $P_\theta(a \mid s)$ over actions and a scalar value estimate $V_\theta(s)$ predicting the expected reward. During MCTS, these outputs guide tree search: the prior biases exploration toward promising actions, while the value estimate enables leaf evaluation without full rollouts. The network architecture uses a shared Graph Attention Network v2 (GATv2) (Brody et al., 2021) backbone with separate actor and critic heads (details in Appendix E). At each decision step t, MCTS builds a search tree rooted at the current state s_t. Each node stores statistics for its valid actions: visit count $N(s, a)$, cumulative value $W(s, a)$, mean action value $Q(s, a) = W(s, a)/N(s, a)$, and prior probability $P(s, a)$ from the policy network.

Selection. Starting from the root, we traverse the tree by selecting actions that maximize the Predictor Upper Confidence bounds applied to Trees (PUCT) criterion (Rosin, 2011):

$$a^* = \underset{a \in \mathcal{C}_t}{\arg\max} \left[Q(s, a) + c \cdot P(s, a) \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \right] \quad (5)$$

where c controls the exploration-exploitation trade-off.

Expansion and Evaluation. When selection reaches an unexpanded node, we query the neural network to obtain $(P, V) = f_\theta(s)$. The prior P is masked to valid actions $\mathcal{C}_t$ and renormalized:

$$P(s, a) = \frac{\exp(\ell_a) \cdot \mathbf{1}\{a \in \mathcal{C}_t\}}{\sum_{b \in \mathcal{C}_t} \exp(\ell_b)}, \quad (6)$$

where ℓ_a is the logit for action a. Instead of simulating to completion to evaluate a newly expanded node, we use the value estimate V directly as the predicted outcome.

Backpropagation. After evaluation, statistics are updated along the path from leaf to root: $N(s, a) \leftarrow N(s, a) + 1$, $W(s, a) \leftarrow W(s, a) + V$, and $Q(s, a) \leftarrow W(s, a)/N(s, a)$.

After N_{iter} simulations, an improved policy $\boldsymbol{\pi}_t$ is derived from visit counts:

$$\boldsymbol{\pi}(a \mid s_t) = \frac{N(s_t, a)^{1/\tau}}{\sum_{b \in \mathcal{C}_t} N(s_t, b)^{1/\tau}}, \quad (7)$$

with temperature τ controlling exploration. At episode end, the reward $z = \mathcal{R}$ is obtained via traffic simulation (Eq. 3). All states in an episode receive the same normalized reward $\tilde{z}$, training the value head to predict expected outcomes. The network parameters are optimized by minimizing:

$$\mathcal{L}(\theta) = -\sum_a \boldsymbol{\pi}(a \mid s) \log P_\theta(a \mid s) + \big(V_\theta(s) - \tilde{z}\big)^2, \quad (8)$$

combining cross-entropy between MCTS visit distributions $\boldsymbol{\pi}$ and network priors P_θ with MSE between normalized returns $\tilde{z}$ and value predictions V_θ.

4. Experiment

Setup. We run all simulations in UXsim (Seo, 2025), a mesoscopic traffic simulator using Newell's car-following model (Newell, 2002). Unlike microscopic alternatives (Lopez et al., 2018), the mesoscopic formulation provides computational efficiency for iterative optimization, running 30–60× faster. The simulator aggregates vehicles into platoons of size $\Delta n = 5$ and advances dynamics with a discrete time step $\Delta t = 1$ second, simulating to a horizon $T_{\text{sim}} = 10{,}000$ steps ($\approx$ 2.7 hours). For transit network design, we extend UXsim with a Gymnasium environment adding a transit layer for bus dispatching and passenger boarding and alighting. It dispatches buses on agent-designed routes with 40 passenger capacity and a 60 s dwell time per stop, and allocates origin–destination demand between bus and car modes via a modal-split parameter α.

We train AlphaTransit on the Bloomington network for 10^6 environment steps using $W = 8$ parallel workers for episode collection, with MCTS performing $N_{\text{iter}} = 400$ simulations per decision step. Each episode constructs a complete 16-route transit network, with each route growing node by node until it reaches the maximum length $L_{\max} = 14$ or until no valid one-hop extension exists. All routes begin at a designated transit center hub, matching the configuration of the existing Bloomington Transit system. We consider two modal-split scenarios: $\alpha = 1.0$, where all served demand uses transit, and $\alpha = 0.3$, a typical urban public transport share that assigns 30% of demand to buses and the remainder to cars (Buehler et al., 2019). This yields two trained AlphaTransit policies, one per demand scenario. Additional hyperparameters are provided in Appendix G Table 2. We compare AlphaTransit against the following baselines:

- *Real-World*: The 16 existing bus routes in the Bloomington Transit network (Bloomington Transit), designed manually using historical data and operational constraints. This serves as a human-designed reference representing current practice.
- *Random Walk*: Samples the next node uniformly from the admissible candidate set $\mathcal{C}_t$.
- *Demand Coverage*: Samples the next node $i \in \mathcal{C}_t$ proportional to its demand interaction with the partial route, $s(i) = \sum_{j \in r_k}(D_{ij} + D_{ji})$.
- *Shortest Path*: Samples the next node $i \in \mathcal{C}_t$ proportional to inverse edge length, $p(i) \propto \exp(-\ell_i)$.
- *Genetic Algorithm (GA)*: A widely used metaheuristic for transit network design (Fan & Machemehl, 2006; Nayeem et al., 2014). Each individual represents a complete route set $\Pi = \{r_1, \ldots, r_K\}$ that evolves over generations through tournament selection, route-exchange crossover, and path-regeneration mutation. Fitness is evaluated using the same simulation-based reward $\mathcal{R}$ (Eq. 3) and frequency-setting rule as all other methods. Full details are provided in Appendix G.
- *End-to-End Reinforcement Learning*: A policy trained with Proximal Policy Optimization (Schulman et al., 2017) selecting actions directly from $\pi_\theta(a \mid s_t)$ without search. To address the long episode horizon and sparse terminal reward, training incorporates reward shaping during route construction and uses generalized advantage estimation (Schulman et al., 2015) for variance reduction. The state representation, terminal reward, and policy network are identical to AlphaTransit. Implementation details are provided in Appendix G.

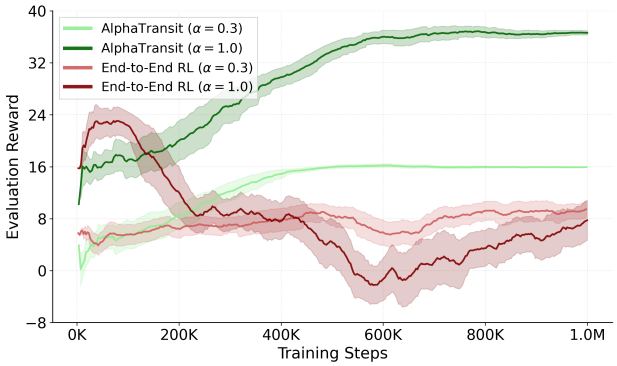


Figure 3. Training curves for AlphaTransit and End-to-End RL under two modal-split scenarios ($\alpha = 0.3$ and $\alpha = 1.0$). Curves show moving averages (window of 50) over training steps, with shaded standard deviation regions. We periodically evaluate each policy by running a full episode and averaging rewards across 5 seeds. AlphaTransit converges faster with greater stability, achieving 8% higher peak rewards under $\alpha = 0.3$ and 14% higher under $\alpha = 1.0$. Notably, End-to-End RL exhibits severe instability under $\alpha = 1.0$: performance collapses from a peak of 32 to below 7 by training's end, due to credit assignment difficulty over the long horizon (up to 16x14 steps) . Each curve shows the best run from 16 independent trials with different seeds.

Performance Metrics. We evaluate designs using seven metrics following established transit frameworks (Vuchic, 2007; 2017). At simulation end, N_{comp}, N_{ongoing}, and N_{waiting} denote passengers who completed trips, are currently traveling, and are waiting at stops, respectively, with $N_{\text{total}} = N_{\text{comp}} + N_{\text{ongoing}} + N_{\text{waiting}}$ denoting all passengers in the transit system. Additionally, N_{boarded} is the number of passengers who boarded buses during the simulation and N_{want} denotes the total number of potential riders.

- *Service rate* (%): Fraction of potential demand that successfully boards, i.e. $N_{\text{boarded}}/N_{\text{want}}$.
- *Wait time* (min): Average waiting time across all riders.
- *Transfer rate* (%): Fraction of completed trips requiring at least one transfer, $N_{\text{transfer}}/N_{\text{comp}} \times 100$.

	Method	Passenger Metrics				Operator Metrics		
		Service Rate (%)	Wait Time (min)	Transfer Rate (%)	Travel Time (min)	Route Efficiency	Fleet Size	Bus Util. (%)
$\alpha = 0.3$	Real-World	42.91 ± 1.22	13.62 ± 0.43	86.23 ± 0.34	46.12 ± 0.85	13.33 ± 0.37	89.00	17.75 ± 0.12
	Random Walk	40.91 ± 1.10	22.32 ± 2.25	85.49 ± 4.42	50.58 ± 1.98	5.38 ± 1.13	37.20 ± 5.49	11.88 ± 0.35
	Demand Coverage	39.72 ± 7.40	20.30 ± 3.74	80.33 ± 2.83	48.29 ± 5.35	6.24 ± 2.06	37.80 ± 8.06	11.13 ± 2.65
	Shortest Path	39.69 ± 4.68	21.86 ± 2.91	78.88 ± 4.76	44.16 ± 4.91	4.62 ± 0.86	$\mathbf{20.60 \pm 1.62}$	6.65 ± 0.79
	Genetic Alg.	53.39 ± 0.93	8.98 ± 0.17	88.44 ± 0.29	36.55 ± 0.49	16.59 ± 0.27	67.00	$\mathbf{19.25 \pm 0.25}$
	End-to-End RL	55.86 ± 1.10	12.68 ± 0.70	87.75 ± 0.17	41.20 ± 0.61	8.03 ± 0.14	46.00	18.44 ± 0.39
	AlphaTransit	$\mathbf{61.34 \pm 0.27}$	$\mathbf{6.90 \pm 0.21}$	$\mathbf{75.86 \pm 0.25}$	$\mathbf{33.63 \pm 0.47}$	$\mathbf{18.15 \pm 0.23}$	55.00	14.66 ± 0.20
$\alpha = 1.0$	Real-World	58.78 ± 1.01	15.99 ± 0.44	81.99 ± 0.30	52.88 ± 0.74	62.00 ± 0.89	281.00	31.77 ± 0.22
	Random Walk	60.39 ± 6.06	15.92 ± 1.68	80.15 ± 3.39	48.73 ± 0.94	32.63 ± 3.25	113.40 ± 14.09	27.13 ± 3.05
	Demand Coverage	65.50 ± 9.37	16.33 ± 3.71	78.63 ± 2.84	50.03 ± 4.55	39.03 ± 9.51	128.20 ± 38.90	29.88 ± 3.86
	Shortest Path	59.55 ± 5.47	20.87 ± 2.62	$\mathbf{70.99 \pm 4.64}$	43.89 ± 2.23	26.32 ± 6.57	$\mathbf{48.60 \pm 11.77}$	19.66 ± 3.18
	Genetic Alg.	81.05 ± 0.55	9.20 ± 0.41	84.81 ± 0.13	44.17 ± 0.54	91.42 ± 0.87	202.00	$\mathbf{44.00 \pm 0.35}$
	End-to-End RL	66.10 ± 0.35	13.42 ± 0.17	82.38 ± 0.17	50.70 ± 0.51	48.35 ± 0.64	193.00	42.11 ± 0.37
	AlphaTransit	$\mathbf{85.50 \pm 0.47}$	$\mathbf{5.95 \pm 0.31}$	86.18 ± 0.09	$\mathbf{35.08 \pm 0.63}$	$\mathbf{91.64 \pm 0.79}$	167.00	30.27 ± 0.30

Table 1. Performance comparison across modal-split scenarios ($\alpha = 0.3$ and $\alpha = 1.0$). Higher is better for service rate, route efficiency, and bus utilization; lower is better for wait time, transfer rate, travel time, and fleet size (bold indicates best). AlphaTransit achieves the best passenger-facing performance in both scenarios (highest service rate and lowest wait/travel times), except for transfer rate under $\alpha = 1.0$, while also attaining the highest route efficiency. Under $\alpha = 0.3$, AlphaTransit additionally achieves the lowest transfer rate (75.9%), serving 61.3% of demand with under 7 minutes average wait time. Under the high-demand $\alpha = 1.0$ setting, AlphaTransit serves 85.5% of passenger demand with an average wait time of under 6 minutes, a 35% reduction compared to the Genetic Algorithm, while deploying 17% fewer buses. Shortest Path heuristic achieves the smallest fleet sizes and the lowest transfer rate under $\alpha = 1.0$, but at the cost of service rates below 60%. Values report mean $\pm$ standard deviation over 10 random seeds (980 total simulation runs). Heuristic baselines (Random Walk, Demand Coverage, Shortest Path) exhibit variance from both stochastic route generation and simulation. In contrast, learning-based methods (GA, End-to-End RL, AlphaTransit) use deterministic evaluation and vary only due to simulation stochasticity, producing identical routes across seeds; hence, fleet size, which depends only on routes, shows no variance on these methods.

- *Travel time* (min): Average total trip duration (wait plus in-vehicle) for passengers who completed trips or are mid-journey,

$$\frac{1}{N_{\text{served}}} \sum_{p} \left(\tau_p^{\text{wait}} + \tau_p^{\text{move}} \right), \tag{9}$$

  where τ_p^{wait} and τ_p^{move} are the waiting and in-vehicle travel times for passenger p.
- *Route efficiency* (pax/km): Passengers served per unit infrastructure, $N_{\text{comp}}/L_{\text{total}}$, where L_{total} is the total route length in kilometers.
- *Fleet size*: Total bus count deployed across all routes.
- *Bus utilization* (%): Average load factor (occupancy / capacity) across all buses.

Results. Table 1 summarizes the performance of all methods under both modal-split scenarios. AlphaTransit consistently outperforms all baselines on key passenger-centric metrics. Under the typical urban modal split ($\alpha = 0.3$), AlphaTransit achieves a service rate of 61.3%, representing a 10% relative improvement over End-to-End RL and a 15% improvement over the Genetic Algorithm (GA), while reducing average wait time to 6.9 minutes, which is 23% lower than GA and 46% lower than End-to-End RL. The improvements are more pronounced under high transit demand ($\alpha = 1.0$): AlphaTransit serves 85.5% of passengers compared to 81.1% for GA and 66.1% for End-to-End RL, while achieving a 35% reduction in wait time (from 9.2 to 6.0 minutes) and a 21% reduction in travel time (from 44.2 to 35.1 minutes) relative to GA.

No single baseline dominates across all metrics. The Shortest Path heuristic achieves the smallest fleet sizes and the lowest transfer rate under $\alpha = 1.0$, but at the cost of service rates below 60%, indicating that minimizing transfers alone does not yield effective network designs. GA achieves the highest bus utilization, reflecting its tendency toward denser route configurations; however, this comes with larger fleet requirements and does not translate to better passenger outcomes. The existing Real-World routes underperform all optimization-based methods on service rate and travel time, highlighting the potential for systematic network redesign. In contrast, AlphaTransit achieves the highest route efficiency while deploying 17–18% fewer buses than GA. Under $\alpha = 0.3$, AlphaTransit additionally achieves the lowest transfer rate, while under $\alpha = 1.0$ the improved passenger efficiency comes with a higher transfer rate and lower bus utilization than GA, reflecting a different operating point that prioritizes low waits and fast end-to-end trips over maximizing load factors.

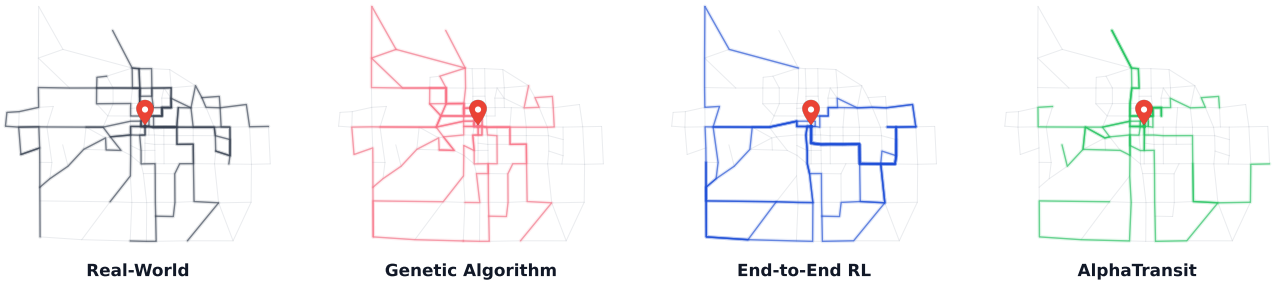


Figure 4. Transit routes from the Real-World benchmark data and learning-based methods for Bloomington, Indiana (best-performing design selected) at $\alpha = 0.3$. Routes appear thicker where multiple services overlap; red pin marks the transit center. Real-World covers 114 nodes (79.7%) with 115.6 km distance but achieves only 42.9% service rate. AlphaTransit uses shortest distance (86.7 km) covering 78 nodes (54.5%) while achieving the highest service rate (61.3%). End-to-End RL consumes the most distance (150.9 km) while covering the fewest nodes (75). Route designs for $\alpha = 1.0$ appear in Appendix G Figure 6.

Route topology in Figure 4 reveals distinct network organization strategies among the methods. Real-World routes maximize geographic spread, utilizing 135 unique edges and 58 potential interchange points, yet this extensive reach does not translate to effective passenger service. AlphaTransit takes a different approach: it uses only 90 edges while achieving the highest service rate, suggesting more targeted resource allocation. While AlphaTransit and Real-World share 64 nodes, each serves distinct areas: 14 nodes exclusive to AlphaTransit and 50 exclusive to Real-World. On the other hand, End-to-End RL exhibits severe redundancy: 48.9% of its edges are shared by multiple routes and 44.0% of served nodes are covered by three or more routes, compared to 22.8% in Real-World. AlphaTransit, by contrast, distributes service more evenly, with 36.7% shared edges and 29.5% of nodes served by three or more routes.

These structural differences translate directly to operational efficiency. When normalized by total route length, AlphaTransit achieves a service-rate-per-kilometer of 0.708 compared to 0.371 for Real-World, a 91% improvement. Similar patterns hold at $\alpha = 1.0$ (see Appendix G Figure 6). This demonstrates that AlphaTransit learns to strategically concentrate routes on high-demand corridors rather than maximizing spatial coverage, achieving the highest service rate with fewer route-kilometers. End-to-end optimization without lookahead produces spatially compact but poorly coordinated networks that fail to account for network-wide effects such as transfer coordination. Similarly, at $\alpha = 0.3$, heuristic baselines (Shortest Path, Demand Coverage, and Random Walk) all achieve service rates around 40%, well below learned methods. Shortest Path minimizes fleet size (21 buses) but yields the lowest route efficiency, while Demand Coverage and Random Walk require larger fleets (37–38 buses) without meaningfully improving coverage.

5. Conclusion

Transit network design is a sequential combinatorial problem where consequences of early decisions emerge only after the network is assembled and simulated. This delayed, nonlocal feedback creates a deceptive optimization landscape: locally attractive choices (such as extending routes toward high-demand corridors or routing multiple corridors through the same transfer hub) can induce transfer bottlenecks, correlated delays, and wasted capacity visible only at deployment. As a result, End-to-End RL methods operating on sparse terminal rewards (or shaping rewards) can converge to local optima that fail to optimize global performance. We presented AlphaTransit, a planning-and-learning framework combining a graph attention policy-value network with Monte Carlo Tree Search, enabling explicit lookahead at each route construction step. By evaluating downstream consequences before committing, AlphaTransit explores decisions that may appear locally suboptimal but enable globally superior network structures once the full route set is constructed and passenger flows adapt.

We introduced a new real-world benchmark derived from Bloomington, Indiana, with accurate road topology, census-based OD demand, and the existing human-designed network as a reference. Across two modal-split regimes ($\alpha = 0.3$ and $\alpha = 1.0$), search-guided learning consistently outperforms End-to-End RL and classical heuristics. Under high transit demand, AlphaTransit achieves 29% higher service rate and reduces average waiting time by 56% relative to End-to-End RL, while deploying fewer buses and nearly doubling route efficiency. Compared to the Real-World network, AlphaTransit improves service rate from 42.91% to 61.34%, reduces wait time by 49%, and reduces travel time by 27%, while using 38% fewer buses. Beyond absolute performance, explicit lookahead changes *which* parts of the design space are explored: whereas End-to-End RL reinforces greedy patterns that maximize immediate demand coverage, MCTS enables exploration of decisions whose short-term reward is negative but whose long-term effect reduces transfer bottlenecks and improves global throughput.

More broadly, the results suggest that search-guided learning can be a practical and effective approach for network

design problems where local signals are misleading and performance is only revealed after complete evaluation. We discuss the limitations and future directions in Appendix H.

Impact Statement

Urban population is projected to reach 60% of the global population by 2030 (United Nations Department of Economic and Social Affairs, 2019), placing unprecedented pressure on transit infrastructure that supports 7.7 billion annual trips in the United States alone (American Public Transportation Association, 2025). In many large cities, less than 20% of jobs can be reached within an hour using affordable transit, disproportionately affecting low-income workers (World Bank, 2024). This work develops automated methods for designing transit route networks that help agencies serve more riders with limited resources, directly addressing service frequency and wait time metrics shown to be the strongest determinants of ridership (TransitCenter, 2019). By optimizing jointly for passenger-facing metrics (coverage, service rate, wait times) and operator constraints (fleet size, utilization), our approach offers a pathway toward more effective transit planning. Our use of real census-derived demand data from Bloomington, Indiana demonstrates applicability to actual planning contexts. However, we acknowledge that optimization objectives may not capture all community priorities, and automated planning should complement rather than replace stakeholder engagement. We encourage future work to incorporate fairness and accessibility constraints to ensure algorithmic transit design supports inclusive mobility goals.

References

Alkilane, K. and Lee, D.-H. MetroZero: Deep reinforcement learning and Monte Carlo tree search for optimized metro network expansion. *IEEE Transactions on Intelligent Transportation Systems*, 26(1):810–823, 2025.

American Public Transportation Association. Public transportation ridership update. Policy brief, APTA, 2025.

Amiri, S., Zehtabi, P., Dervovic, D., and Cashmore, M. Surrogate assisted monte carlo tree search in combinatorial optimization. *arXiv preprint arXiv:2403.09925*, 2024.

Bello, I., Pham, H., Le, Q. V., Norouzi, M., and Bengio, S. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.

Bloomington Transit. Gtfs schedule dataset. https://bloomingtontransit.com/gtfs/.

Brody, S., Alon, U., and Yahav, E. How attentive are graph attention networks? *arXiv preprint arXiv:2105.14491*, 2021.

Buehler, R., Pucher, J., and Dümmler, O. Verkehrsverbund: The evolution and spread of fully integrated regional public transport in germany, austria, and switzerland. *International Journal of Sustainable Transportation*, 13(1): 36–50, 2019.

Ceder, A. *Public transit planning and operation: Modeling, practice and behavior*. CRC press, 2016.

Coholich, J. A bag of tricks for deep reinforcement learning, 2023. URL https://www.jeremiahcoholich.com/post/rl_bag_of_tricks/#observation-normalization-and-clipping.

Darvariu, V.-A., Hailes, S., and Musolesi, M. Planning spatial networks with Monte Carlo tree search. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 479(2269):20220383, 2023.

Darwish, A., Khalil, M., and Badawi, K. Optimising public bus transit networks using deep reinforcement learning. In *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, pp. 1–7. IEEE, 2020.

Durán-Micco, J. and Vansteenwegen, P. A survey on the transit network design and frequency setting problem. *Public transport*, 14(1):155–190, 2022.

Fan, W. *Optimal transit route network design problem: Algorithms, implementations, and numerical results*. The University of Texas at Austin, 2004.

Fan, W. and Machemehl, R. B. Optimal transit route network design problem with variable transit demand: genetic algorithm approach. *Journal of transportation engineering*, 132(1):40–51, 2006.

Furth, P. G. and Wilson, N. H. Setting frequencies on bus routes: Theory and practice. *Transportation Research Record*, 818(1981):1–7, 1981.

Heyken Soares, P., Mumford, C. L., Amponsah, K., and Mao, Y. An adaptive scaled network for public transport route optimisation. *Public Transport*, 11(2):379–412, 2019.

Holliday, A. and Dudek, G. Augmenting transit network design algorithms with deep learning. In *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*, pp. 2343–2350. IEEE, 2023.

Holliday, A. and Dudek, G. A neural-evolutionary algorithm for autonomous transit network design. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4457–4464. IEEE, 2024.

Holliday, A., El-Geneidy, A., and Dudek, G. Learning heuristics for transit network design and improvement

with deep reinforcement learning. *Transportmetrica B: Transport Dynamics*, 13(1):2561863, 2025.

Huang, S. and Ontañón, S. A closer look at invalid action masking in policy gradient algorithms. *arXiv preprint arXiv:2006.14171*, 2020.

Huang, S., Dossa, R. F. J., Raffin, A., Kanervisto, A., and Wang, W. The 37 implementation details of proximal policy optimization. *The ICLR Blog Track 2023*, 2022.

Kemmerling, M., Lütticke, D., and Schmitt, R. H. Beyond games: a systematic review of neural monte carlo tree search applications. *arXiv preprint arXiv:2303.08060*, 2023.

Kepaptsoglou, K. and Karlaftis, M. Transit route network design problem. *Journal of transportation engineering*, 135(8):491–505, 2009.

Khalil, E., Dai, H., Zhang, Y., Dilkina, B., and Song, L. Learning combinatorial optimization algorithms over graphs. *Advances in neural information processing systems*, 30, 2017.

Li, J., Dong, H., Zhao, X., Tang, H., Yin, A., and Xue, R. A transit network design and frequency setting model with graph neural network and deep reinforcement learning. In *Sixth International Conference on Computer Information Science and Application Technology (CISAT 2023)*, volume 12800, pp. 128005Y. SPIE, 2023.

Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., and Wießner, E. Microscopic traffic simulation using SUMO. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, pp. 2575–2582. IEEE, November 2018.

Mandl, C. E. Evaluation and optimization of urban public transportation networks. *European Journal of Operational Research*, 5(6):396–404, 1980.

McGuckin, N., Fucci, A., et al. Summary of travel trends: 2017 national household travel survey. Technical report, United States. Department of Transportation. Federal Highway Administration, 2018.

Mumford, C. L. New heuristic and evolutionary operators for the multi-objective urban transit routing problem. In *2013 IEEE congress on evolutionary computation*, pp. 939–946. IEEE, 2013.

Nayeem, M. A., Rahman, M. K., and Rahman, M. S. Transit network design by genetic algorithm with elitism. *Transportation Research Part C: Emerging Technologies*, 46: 30–45, 2014.

Newell, G. F. Some issues relating to the optimal design of bus routes. *Transportation Science*, 13(1):20–35, 1979.

Newell, G. F. A simplified car-following theory: a lower order model. *Transportation Research Part B: Methodological*, 36(3):195–205, 2002.

Nikolić, M. and Teodorović, D. Transit network design by bee colony optimization. *Expert Systems with Applications*, 40(15):5945–5955, 2013.

Owais, M. Transit network design problem: a half century of methodological research. *Innovative Infrastructure Solutions*, 11(1):3, 2026.

Roess, R., Prassas, E., and McShane, W. *Traffic Engineering*. Pearson, 2011. ISBN 9780136135739. URL https://books.google.com/books?id=fGE6PgAACAAJ.

Rosin, C. D. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3): 203–230, 2011.

Schmidt, M. and Schöbel, A. Planning and optimizing transit lines. *arXiv preprint arXiv:2405.10074*, 2024.

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., Lillicrap, T., and Silver, D. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020. doi: 10.1038/s41586-020-03051-4.

Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Seo, T. Uxsim: lightweight mesoscopic traffic flow simulator in pure python. *Journal of Open Source Software*, 10 (106):7617, 2025.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362 (6419):1140–1144, 2018.

Sutton, R. S. and Barto, A. G. *Reinforcement learning: An introduction*. MIT press, 2018.

TransitCenter. Who's on board 2019: How to win back america's transit riders. Technical report, TransitCenter, New York, 2019.

Transitland. Bloomington transit (bt) — operator details. https://www.transit.land/operators/o-dnfq-bloomingtontransit.

United Nations Department of Economic and Social Affairs. World urbanization prospects: The 2018 revision. Technical report, United Nations, 2019. URL https://population.un.org/wup/.

U.S. Census Bureau. Tiger/line shapefiles. https://www.census.gov/cgi-bin/geo/shapefiles/index.php, 2025. Accessed: 2025-10-06.

U.S. Census Bureau, Center for Economic Studies. Lehd origin-destination employment statistics (lodes). https://lehd.ces.census.gov/data, 2025. Accessed: 2025-10-06.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., and Bengio, Y. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

Vermeir, E., Engelen, W., Philips, J., and Vansteenwegen, P. An exact solution approach for the bus line planning problem with integrated passenger routing. *Journal of Advanced Transportation*, 2021(1):6684795, 2021.

Vinyals, O., Fortunato, M., and Jaitly, N. Pointer networks. *Advances in neural information processing systems*, 28, 2015.

Vuchic, V. R. *Urban transit systems and technology*. John Wiley & Sons, 2007.

Vuchic, V. R. *Urban transit: operations, planning, and economics*. John Wiley & Sons, 2017.

Weng, D., Chen, R., Zhang, J., Bao, J., Zheng, Y., and Wu, Y. Pareto-optimal transit route planning with multi-objective monte-carlo tree search. *IEEE Transactions on Intelligent Transportation Systems*, 22(2):1185–1195, 2020.

World Bank. Promoting livable cities by investing in urban mobility. Results brief, World Bank Group, 2024.

Yang, T., Wang, Y., Yue, Z., Yang, Y., Tong, Y., and Bai, J. Graph pointer neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 36, pp. 8832–8839, 2022.

Yoo, S., Lee, J. B., and Han, H. A reinforcement learning approach for bus network design and frequency setting optimisation. *Public Transport*, 15(2):503–534, 2023.

Zhao, F. and Zeng, X. Simulated annealing–genetic algorithm for transit network optimization. *Journal of Computing in Civil Engineering*, 20(1):57–68, 2006.

A. Definitions

We formalize the components of the transit network and the design process below:

Definition 1 (Road Network)**.** A road network is an undirected graph $G = (V, E)$ where $V = \{v_1, \ldots, v_n\}$ is the set of n nodes representing intersections or stops, and $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ is the set of edges representing bidirectional road segments. Each edge $e = \{u, v\} \in E$ has an associated length $\ell_e > 0$ and free-flow speed $s_e > 0$. The neighborhood of node v is $\mathcal{N}(v) = \{u \in V \mid \{u, v\} \in E\}$.

Definition 2 (Transit Route)**.** A transit route r_k is a simple path in G defined as an ordered sequence of distinct nodes:

$$r_k = (v_{k,1}, v_{k,2}, \ldots, v_{k,\ell_k})$$

where $\ell_k \geq 2$ is the route length, $\{v_{k,i}, v_{k,i+1}\} \in E$ for all $i \in \{1, \ldots, \ell_k - 1\}$, and $v_{k,i} \neq v_{k,j}$ for $i \neq j$. Routes are bidirectional, with vehicles traversing both $v_{k,1} \to v_{k,\ell_k}$ and $v_{k,\ell_k} \to v_{k,1}$.

Definition 3 (Transit Network)**.** A transit network $\Pi = \{r_1, \ldots, r_K\}$ is a collection of K routes operating on G. The network must satisfy constraints such as route length: $L_{\min} \leq |r_k| \leq L_{\max}$ for all $k \in \{1, \ldots, K\}$.

Definition 4 (Stops and Frequency of Service). For route r_k with stop spacing parameter $s_k \geq 1$, the stop set is:

$$\mathcal{S}(r_k) = \{v_{k,1+js_k} \mid j = 0, 1, \ldots, \lfloor (\ell_k - 1)/s_k \rfloor\}$$

Each route operates with frequency $\mathcal{F}_k \in \mathbb{N}_{>0}$ vehicles per hour in each direction. The tuple $(r_k, s_k, \mathcal{F}_k)$ fully specifies the operational characteristics of route k.

Definition 5 (Origin-Destination Demand)**.** The travel demand is represented by matrix $D \in \mathbb{R}_{\geq 0}^{n \times n}$ where D_{ij} denotes passenger flow (trips/hour) from origin i to destination j. The modal split parameter $\alpha \in [0, 1]$ determines the fraction of demand using transit, yielding effective transit demand $D_{ij}^{\text{transit}} = \alpha \cdot D_{ij}$.

Definition 6 (Sequential Route Construction)**.** Routes are constructed incrementally through a sequence of decisions. At step t, the partial route is $r^{(t)} = (v_1, \ldots, v_t)$ with frontier node v_t. The admissible extension set is:

$$\mathcal{C}_t = \{u \in \mathcal{N}(v_t) \mid u \notin \{v_1, \ldots, v_t\}\}$$

representing valid next nodes to maintain a simple path.

Definition 7 (Transfer)**.** A transfer occurs when a passenger journey from origin i to destination j requires traversing multiple routes. Let $\mathcal{P}_{ij}$ denote the set of feasible paths from i to j using the transit network. A path $p \in \mathcal{P}_{ij}$ requiring $m > 1$ distinct routes incurs $m - 1$ transfers.

Definition 8 (Transit Route Network Design Problem (TRNDP))**.** Given a road network $G = (V, E)$, an origin–destination (OD) demand matrix $D \in \mathbb{R}_{\geq 0}^{|V| \times |V|}$, a prescribed number of routes $K \in \mathbb{N}_{>0}$, and route-length bounds $L_{\min}, L_{\max} \in \mathbb{N}$ with $2 \leq L_{\min} \leq L_{\max}$, the *Transit Route Network Design Problem* is to choose an operated transit network

$$(\Pi, \mathbf{s}, \boldsymbol{\mathcal{F}}) \;=\; \big(\{r_1, \ldots, r_K\},\, (s_1, \ldots, s_K),\, (\mathcal{F}_1, \ldots, \mathcal{F}_K)\big)$$

that optimizes a system-level performance criterion under demand D. A design $(\Pi, \mathbf{s}, \boldsymbol{\mathcal{F}})$ is *feasible* if for every $k \in \{1, \ldots, K\}$: (i) r_k is a bidirectional simple path in G (Definition 2), (ii) $L_{\min} \leq |r_k| \leq L_{\max}$, (iii) $s_k \in \mathbb{N}_{\geq 1}$ induces the stop set $\mathcal{S}(r_k)$ (Definition 4), (iv) $\mathcal{F}_k \in \mathbb{N}_{>0}$ is the service frequency (vehicles/hour per direction), and any additional operational constraints (e.g., fleet or budget limits) are satisfied.

Let $\mathcal{R}(\Pi, \mathbf{s}, \boldsymbol{\mathcal{F}})$ denote a scalar objective that evaluates passenger and operator performance of the operated network (e.g., the simulation-based reward in Eq. (3)); if the evaluation is stochastic, $\mathcal{R}$ is understood as a random variable induced by passenger arrivals, routing, and simulation dynamics. The TRNDP is the optimization problem

$$(\Pi^\star, \mathbf{s}^\star, \boldsymbol{\mathcal{F}}^\star) \in \arg\max_{(\Pi, \mathbf{s}, \boldsymbol{\mathcal{F}}) \in \mathcal{X}} \mathbb{E}\big[\mathcal{R}\big(\Pi, \mathbf{s}, \boldsymbol{\mathcal{F}}\big)\big]\,,$$

where $\mathcal{X}$ is the set of all feasible designs.

In this work, we fix stop spacing to $s_k = 1$ for all k and set frequencies deterministically via the max-load rule in Appendix D, i.e., $\boldsymbol{\mathcal{F}} = \mathrm{FOS}(\Pi)$; thus the decision variables reduce to the route set Π.

Helper functions are used in Algorithms 1 and 2 are:

Definition 9 (Mask)**.** $\textsc{Mask}(\mathcal{C}, n)$ constructs a binary mask $m \in \{0, 1\}^n$ over all nodes, where $m[i] = 1$ if $i \in \mathcal{C}$ and $m[i] = 0$ otherwise. This restricts the policy to sample only from valid candidate actions.

Definition 10 (State Construction)**.** $\textsc{FormState}$ constructs node features X_t encoding current route nodes V_{cur}, completed route nodes V_{cmp}, and demand patterns as defined in Appendix F.

B. Real-world Network and Data Processing

We introduce a novel city-scale transit network dataset for Bloomington, Indiana. Unlike existing benchmark networks from literature, our dataset uniquely captures real-world aspects in three dimensions: (i) the underlying transportation network, (ii) travel demand derived from census data, and (iii) transit routes currently operating in the city.

Network Structure. The network consists of 143 nodes and 243 bidirectional edges, covering an area of approximately 152.3 km^2. The network topology was derived from road infrastructure with several practical assumptions made to balance modeling fidelity with simulation efficiency.

- *2D Representation*: 3D infrastructure elements such as tunnels, overpasses, and underpasses are modeled as planar (2D) connections.
- *Edge Geometry*: All edges are represented as bidirectional. When two parallel one-way streets exist next to each other, they are consolidated into single a bidirectional edge positioned at their centerline. Further, edges follow shortest-distance connections between nodes rather than exact street curvatures; the length differences are negligible for the scale of analysis.
- *Speed Assignment*: All edges are assigned a uniform free-flow speed of 16.67 m/s (60 km/h or 37 miles/hr), reflecting typical urban traffic speed limit.
- *Highway Exclusion*: Interstate 69 highway segments within the city limits were excluded from the transit network, as city transit primarily serves local destinations and highways lack appropriate passenger access points. The bus routes currently operating in the city also do not utilize the highway.
- *Shared Routes*: Although existing bus routes may have slightly different paths for outbound and inbound directions, to reduce complexity, we model them as identical paths, manually selecting the most appropriate nodes (based on access, length, and community served) to maintain essential connectivity.

Coordinate Transformation. The raw geospatial data uses geographic coordinates (latitude and longitude) in angular units. However, operations in our processing pipeline such as calculating census block centroids and edge distances require a flat, Cartesian plane for accurate results. Therefore, the coordinates were transformed to a Cartesian coordinate system using the Universal Transverse Mercator (UTM) Zone 16N projection, which covers Indiana including Bloomington. The UTM projection preserves local angles and shapes while providing true metric distances and areas with minimal distortion, making it ideal for our case.

Demand Generation. The primary demand component was obtained from commuting trips captured in the 2022 LEHD Origin-Destination Employment Statistics (LODES) from the U.S. Census Bureau (U.S. Census Bureau, Center for Economic Studies, 2025), which provides flows between census blocks with home locations as origins and work locations as destinations. We utilize block-level census data (U.S. Census Bureau, 2025) and processed a total of $2,399$ census blocks within Monroe County, Indiana (FIPS code 105, GEOID 18105), which was further reduced to $1,475$ blocks to confine the area of interest to the vicinity of Bloomington. For each census block, its centroid was calculated and the demand origins and destinations in that block were assigned to the node nearest to the centroid.

Because the LODES data excludes trips for non-commuting purposes such as school and shopping (typically classified as home-based other or non-home-based in transportation research), to account for this mixed composition of traffic, we scale the commuting flows by 150%, consistent with typical values ranging from $100\% - 200\%$ depending on time of day (McGuckin et al., 2018). Further, to express demand on an hourly basis, we adopt a peak-hour share of 11% of daily traffic, which lies within the typical $6\% - 12\%$ range (Roess et al., 2011). The resulting origin–destination (OD) demand matrix contains $5,738$ pairs, with a maximum origin demand of 344 vehicles per hour and a maximum destination demand of $1,681$ vehicles per hour.

Existing Bus Routes. The Bloomington Transit system (Bloomington Transit; Transitland) operates 16 bus routes which map to an average of 14.2 nodes per route in our network (ranging from 8 to 24 nodes).

C. Size of the search space

The scope of the design problem is to choose $R = 16$ routes, each visiting 14 distinct nodes, on the Bloomington network graph $G = (V, E)$. Computing the exact size of the search space is computationally intensive, so we estimate it by approximating the number of simple paths (no repeated nodes) consisting of $L = 13$ edges. The G has $|V| = 143$ and $|E| = 243$, which gives average degree

$$d \;=\; \frac{2|E|}{|V|} \;=\; \frac{486}{143} \;\approx\; 3.40.$$

For simple paths in sparse graphs, we approximate the number of candidate routes by accounting for the constraint that nodes cannot repeat. Starting from any of $|V|$ nodes, the first step has approximately d choices, and each subsequent step has approximately $d - 1$ choices (i.e., excluding the node just visited):

$$P \;\approx\; |V| \cdot d \cdot (d-1)^{L-1}.$$

Treating a design as an ordered list of $R = 16$ routes, the total search space is

$$S \;=\; P^R \;\approx\; \big(|V| \cdot d \cdot (d-1)^{L-1}\big)^{16}.$$

Numerically,

$$(d-1)^{12} = (2.40)^{12} \approx 3.65 \times 10^4,\; P \approx 1.78 \times 10^7,$$

$$S \approx 9.5 \times 10^{115}.$$

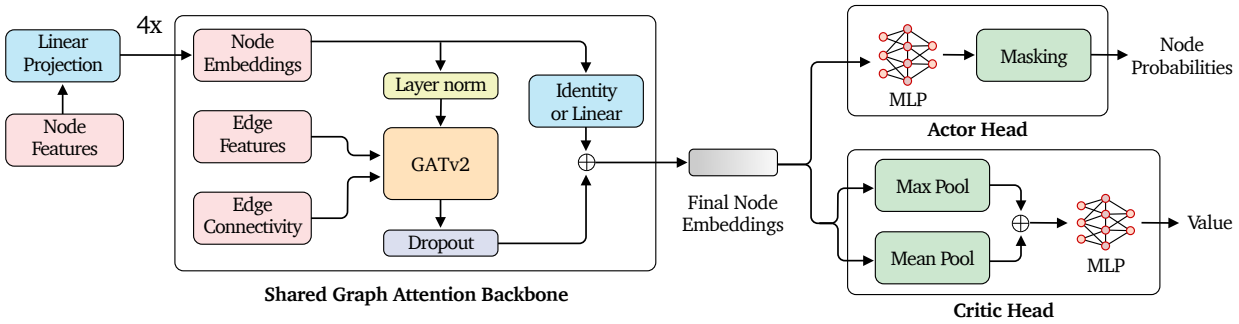


Figure 5. Overview of the policy network. A shared graph attention backbone maps 16 node features, 2 edge features, and edge connectivity to final node embeddings of size 64. First, the node features are projected to 64 channels. Then the representation passes through a stack of four GATv2 blocks in sequence with the output of attention heads averaged (instead of concatenated) to control channel growth. Each block uses pre-layer normalization, a GATv2 attention layer with dropout, and a residual path that copies the input when widths match or applies a linear projection when they differ. The **actor head** is an MLP with three hidden layers of widths 256, 128, and 64 that produces a score per node, followed by a feasibility mask to obtain probabilities over valid nodes. The **critic head** concatenates global mean and global max pooling of the node embeddings, then feeds the result to an MLP with three hidden layers of widths 256, 128, and 64 to predict a scalar value per graph.

The magnitude of S makes exhaustive or near-exhaustive search infeasible, motivating alternative approaches that exploit the problem's structure. Note that this approximation does not adjust for overlapping edges between routes.

D. Frequency of Service assignment

Frequency of Service (FOS) setting is integral to the Transit Route Network Design Problem, as it directly affects both passenger satisfaction (via wait and travel times) and operator costs (e.g., fleet size). FOS measures the number of buses dispatched per hour on a route to maintain adequate service. Learning FOS on a sequential route design (either jointly during route construction or at episode end) yields inconsistent or mis-guided learning signals. For example, during initial stages of the route construction process, the route might just be two nodes $[1 \rightarrow 5]$, this short segment only serves a single O-D pair. The agent may learn that setting a high frequency is beneficial for reducing wait times, etc. But later, when routes are longer, high frequencies might induce over-servicing and increased costs; i.e., a frequency of 6 buses/hour is a bad choice for a 2-node route but might be an excellent choice for a 10-node route. This inconsistency makes it challenging for the an agent to learn a coherent policy, as it would have to adapt to completely different frequency preferences at each step of route construction. We instead compute FOS deterministically post-route design using a max-load principle (Ceder, 2016; Furth & Wilson, 1981). For each route k, frequency is set to ensure the peak passenger load on the busiest segment does not exceed comfortable capacity:

$$\mathcal{F}_k = \left\lceil \frac{\mathcal{Q}k, \max^{\text{norm}}}{\delta_{\max} \cdot \mathcal{C}_k} \right\rceil \tag{10}$$

where $\mathcal{F}_k$ is the frequency for route k, $\mathcal{Q}^{\text{norm}}_{k,\max}$ is the maximum normalized segment load, $\delta_{\max}$ is the comfort threshold (maximum desired load factor, e.g., 0.8 for 80% capacity), and $\mathcal{C}_k$ is bus capacity. To compute $\mathcal{Q}^{\text{norm}}_{k,\max}$, we account for both direct trips (origin and destination on route k) and transfer trips (using route k as part of a multi-route journey). During normalization, we divide each segment's load by the number of overlapping routes to prevent overestimation of frequency requirements when multiple routes serve the same segment. The maximum normalized load across all segments of route k determines $\mathcal{F}_k$. While the RL agent has no direct control over frequency assignment, it indirectly influences it through its route design decisions: for example, routes serving high-demand corridors receive higher frequencies, whereas when multiple routes overlap a segment, the normalized load decreases and each route receives proportionally lower frequency.

E. Policy Network

The policy network consists of a shared graph attention backbone and two separate heads for the actor and critic, as shown in Figure 5. To make the policy scalable across cities of different sizes, we design the architecture so that the number of trainable parameters is independent of the node count n, while the forward computational cost grows linearly with it. This is achieved by: (i) using a shared Graph Attention Networks v2 (GATv2) (Brody et al., 2021) encoder that produces fixed-width node embeddings for any graph size, (ii) implementing an actor that assigns a score to each node with one shared MLP. For the critic, we apply mean and max pooling to the final node embeddings and concatenate them before passing as input. Together, these design choices keep the actor permutation equivariant, the critic permutation invariant i.e., ensuring the policy remains stable under node reindexing, and the parameter count independent of the number of nodes n in the input graph.

Shared GAT backbone. At time t, the input is $(X_t, \mathcal{I}, Z)$ where $X_t \in \mathbb{R}^{n\times 16}$ are node features, $\mathcal{I}$ represents edge connectivity as a directed edge list, and $Z \in \mathbb{R}^{|\mathcal{I}|\times 2}$ are edge attributes. A linear projection maps node features to 64 channels, then a stack of four GATv2 blocks produces final node embeddings

$$H = \text{GATv2Stack}\big(\text{Proj}(X_t), \mathcal{I}, Z\big) \in \mathbb{R}^{n\times 64}.$$

Compared with the original GAT (Veličković et al., 2017), GATv2 employs a dynamic attention mechanism that depends jointly and nonlinearly on both endpoints and is strictly more expressive (Brody et al., 2021). It also conditions on edge attributes, so the edge features (length and free-flow speed) inform message passing and improve modeling of link-dependent interactions. Each block applies pre-layer normalization, a GATv2 attention layer with averaged heads, attention and feature dropout, and an identity skip connection when input and output widths match, or a linear projection when they differ.Block 1 outputs width 128 with 8 heads and uses a projection on the skip ($64\to 128$); Block 2 outputs width 128 with 8 heads and keeps identity ($128 \to 128$); Block 3 outputs width 64 with 4 heads and uses a projection ($128\to 64$); Block 4 outputs width 64 with 4 heads and keeps identity ($64\to 64$).

Actor Head. The actor uses a pointer-style scoring mechanism (Vinyals et al., 2015; Yang et al., 2022). A shared multi-layer perceptron (MLP) scores all nodes in parallel, applies a feasibility mask restricting the distribution to admissible candidates, and selects exactly one node per step. Unlike decoder-based pointer networks that use a recurrence to emit an ordered sequence of pointers, our single-step selection naturally aligns with sequential route construction where decisions are made one node at a time. The actor MLP consists of three hidden layers of widths 256, 128, and 64 that produces a scalar score per node, with weights shared across nodes. A feasibility mask from the environment restricts probability mass only to the admissible candidates that extend the current route. Node features X_t include frontier membership and valid next indicators, so the scorer implicitly conditions on the evolving route context.

Critic Head. The critic concatenates global mean and global max pooling of the node embeddings, then uses an MLP with three hidden layers of widths 256, 128, and 64 to predict a scalar value $V(G)$ per graph.

The backbone is computed once per graph to obtain H, after which the actor produces per-node scores in a single pass and the critic aggregates with constant-time pooling. This yields a forward pass that scales as $\mathcal{O}(B|\mathcal{I}|d + nd)$ for B blocks of width d. Implementation-wise, all linear layers use orthogonal initialization, and dropout is applied on attention coefficients and block outputs in the backbone, as well as in the hidden layers of both heads for regularization.

F. State Representation

The state (s_t) encodes the static network and evolving design context on G with $s_t = \big(X_t,\ \mathcal{I},\ Z\big)$.

Node features ($X_t \in \mathbb{R}^{n\times 16}$). Let V_{cur} be the nodes already placed on the route under construction at time t, V_{cmp} the union of nodes across completed routes, and $V_{\text{core}} = V_{\text{cur}} \cup V_{\text{cmp}}$. Let $\mathcal{C}_t$ be the set of admissible candidates at time t, i.e., the one-hop neighbors of the current frontier that are not already in the route V_{cur}. The node features for node i can then be classified into six groups:

- Geometry and connectivity: normalized (x_i, y_i) coordinates and node degree.
- OD marginals:

$$d_{\text{out}}(i) = \sum_{j=1}^{n} D_{ij}, \qquad d_{\text{in}}(i) = \sum_{j=1}^{n} D_{ji}. \tag{11}$$

- Candidate demand between i and the current route (nonzero only if $i \in \mathcal{C}_t$):

$$a^{\text{cand}}_{i\to\text{cur}} = \mathbf{1}\{i \in \mathcal{C}_t\} \sum_{j\in V_{\text{cur}}} D_{ij}, \tag{12}$$

$$a^{\text{cand}}_{i\leftarrow\text{cur}} = \mathbf{1}\{i \in \mathcal{C}_t\} \sum_{j\in V_{\text{cur}}} D_{ji}. \tag{13}$$

- Designed network demand at i with respect to nodes already in any designed route (nonzero only if $i \in V_{\text{core}}$):

$$a^{\text{core}}_{i\to\text{core}} = \mathbf{1}\{i \in V_{\text{core}}\} \sum_{j\in V_{\text{core}}} D_{ij}, \tag{14}$$

$$a^{\text{core}}_{i\leftarrow\text{core}} = \mathbf{1}\{i \in V_{\text{core}}\} \sum_{j\in V_{\text{core}}} D_{ji}. \tag{15}$$

- Route conditioned demand for all nodes:

$$a^{\text{all}}_{i\to\text{cur}} = \sum_{j\in V_{\text{cur}}} D_{ij}, \quad a^{\text{all}}_{i\leftarrow\text{cur}} = \sum_{j\in V_{\text{cur}}} D_{ji}, \tag{16}$$

$$a^{\text{all}}_{i\to\text{cmp}} = \sum_{j\in V_{\text{cmp}}} D_{ij}, \quad a^{\text{all}}_{i\leftarrow\text{cmp}} = \sum_{j\in V_{\text{cmp}}} D_{ji}. \tag{17}$$

- Flags: one indicator each for current route membership $\mathbf{1}\{i \in V_{\text{cur}}\}$, the fraction of completed routes that contain i in $[0, 1]$, and valid next node $\mathbf{1}\{i \in \mathcal{C}_t\}$.

This representation exposes the policy to demand patterns at multiple scales, from immediate next candidate nodes to the

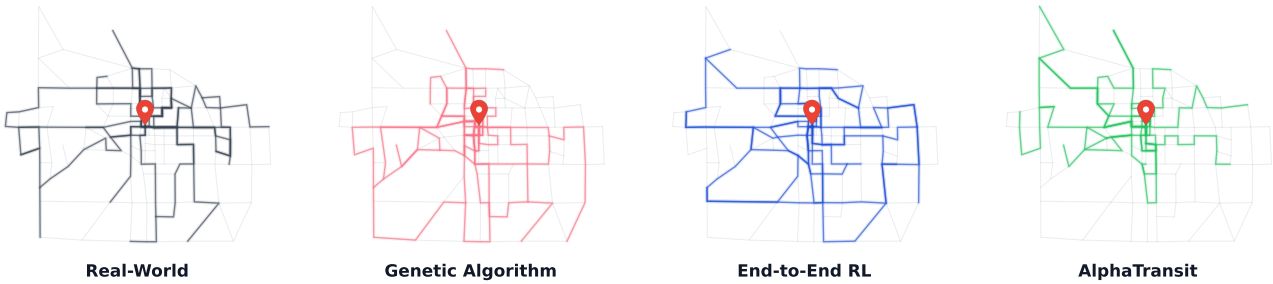


Figure 6. Transit routes from the Real-World and those designed by learning-based methods (best-performing design selected) at $\alpha = 1.0$. Routes appear thicker where multiple services overlap. The Real-World uses 115.57 km of routes covering 114 nodes (79.72%), achieving a 58.78% service rate. End-to-End RL requires 130.42 km to cover 107 nodes (74.83%) with a 66.10% service rate. AlphaTransit attains an 85.50% service rate with 93.21 km total length, covering 97 nodes (67.83% of the network). By concentrating routes on high-demand corridors, AlphaTransit achieves superior service rates with more efficient resource utilization.

global network, enabling it to reason about ridership potential, transfers, and coverage, without requiring direct access to the OD matrix. Equations (11) summarize the baseline incoming and outgoing demands at node i, independent of the current design. Equations (12) and (13) quantify the immediate marginal gain of selecting a candidate by measuring flows between a candidate and the current route. Equations (14) and (15) summarize how well a node is integrated with the already designed network. The route conditioned terms (16) and (17) provide additional context i.e., a global view of demand relative to the current and completed routes.

Edge Connectivity ($\mathcal{I}$). is the directed edge list that encodes graph topology for message passing. Since streets are bidirectional, we include both (u, v) and (v, u) i.e., $\mathcal{I} = \{(u, v) \in V \times V \mid \{u, v\} \in E\}$.

Edge features ($Z \in \mathbb{R}^{|\mathcal{I}| \times 2}$). For each directed edge $(u, v) \in \mathcal{I}$, Z stores a row with length and free flow speed.

All continuous features are min-max scaled to $[0, 1]$ using network level bounds. Demand aggregates are computed from D and scaled by the global reference $\max\big(\max_i \sum_j D_{ij},\ \max_j \sum_i D_{ji}\big)$. Terms that depend on $\mathcal{C}_t$ or V_{core} are set to zero outside those sets. Binary indicators take values in $\{0, 1\}$.

G. Training Procedures and Hyperparameters

All experiments use the same policy network architecture (Appendix Section E) and were conducted on an Intel Core i9-14900K processor with an NVIDIA RTX PRO 6000 GPU. PPO training runs for roughly 60 hours, while AlphaTransit runs for approximately 72 hours. Table 2 summarizes the hyperparameters for both methods.

Genetic Algorithm. We implement a genetic algorithm for transit network design where each individual represents a complete transit network $\Pi = \{r_1, \ldots, r_K\}$ of K routes, each satisfying $L_{\min} \leq |r_k| \leq L_{\max}$.

- *Initialization*: To prevent premature convergence to local optima, the initial population combines three seeding strategies: (i) demand-guided construction that greedily extends routes toward high OD-interaction nodes, (ii) random feasible routes via constrained random walks, and (iii) a warm-start individual sampled from the existing real-world routes.
- *Operators*: Selection uses tournament selection with size 3. Crossover performs route exchange where each route index inherits from one parent with equal probability, preserving complete routes to maintain feasibility. Mutation applies path regeneration by cutting a route at a random interior node and regrowing via random walk, preserving promising prefixes while exploring new suffixes. A repair step ensures all routes satisfy $L_{\min}$ after each operation.
- *Fitness*: Each individual is evaluated by assigning frequencies via the max-load rule (Appendix D) and running a full simulation. Fitness uses the same terminal-only reward $\mathcal{R}$ (Eq. 3) as AlphaTransit.

We use population size 50 with crossover rate 80%, mutation rate 40%, and elitism count 5, yielding approximately 5,000 simulator evaluations (approximately 1M environment steps) and a total wall-clock runtime of 8 hours. We run for 100 generations, with GA $\alpha = 0.3$ finding its best at generation 94 and GA $\alpha = 1.0$ at generation 81.

End-to-End RL. A key challenge in applying standard policy gradient methods to the Transit Route Network Design Problem (TRNDP) is the long episode horizon: with $K = 16$ routes of up to $L_{\max} = 14$ nodes each, episodes can span over 200 decisions before any terminal reward is available. This creates a severe credit assignment problem (Sutton & Barto, 2018), as the policy must learn which early decisions contributed to the final outcome. To address this, the end-to-end approach augments the reward from Eq. (3) with lightweight shaping rewards during route construction. These provide more frequent feedback to guide learning, while the simulation-based reward remains the

primary signal. The full training procedure is summarized in Algorithm 2, where V_{cur} denotes nodes in the current route and V_{cmp} denotes nodes across all completed routes, with supporting functions defined in Appendix A.

During the construction of a route r_k, the agent receives feedback based on the evolving network topology. We use the incremental change in demand coverage $\Delta\Psi_t = \max(0, \Psi_t - \Psi_{t-1})$ to reward actions that expand the reachable O-D pairs. This delta-based formulation directly measures the value of each action rather than accumulated state, providing clearer credit assignment. Additionally, we penalize route overlap and early termination:

$$\begin{aligned}\mathcal{R}_{\text{partial}} &= b_7 \cdot \Delta\Psi_t + b_8 \cdot \omega \\ &\quad + b_9 \cdot \left(1 - \frac{|r_k|}{L_{\max}}\right) \mathbf{1}\{\text{forced end}\},\end{aligned} \tag{18}$$

where ω is the route overlap ratio. The forced end penalty applies when a route terminates before reaching $L_{\max}$ due to no valid extensions, with the penalty proportional to the shortfall. We use $b_7 = 20$, $b_8 = -8$, and $b_9 = -15$, chosen through Bayesian hyperparameter optimization, maximizing mean evaluation reward over 5 validation seeds. The partial reward coefficients are deliberately small relative to the simulation-based reward (Eq. (3)) to ensure the primary learning signal comes from actual passenger flow outcomes. Partial rewards are also normalized online during training.

AlphaTransit. Training alternates between parallel data generation and network optimization. During data generation, W workers independently construct complete transit networks using Monte Carlo Tree Search guided decisions. At each step, Dirichlet noise is added to the root priors to encourage exploration:

$$P(s,a) \leftarrow (1-\varepsilon)\cdot P(s,a) + \varepsilon \cdot \eta_a, \quad \eta \sim \text{Dir}(\alpha_{\text{dir}}), \tag{19}$$

where α_{dir} controls noise concentration and ε controls the noise weight. The search tree persists across the entire episode, with re-rooting to the selected child after each action. This preserves accumulated search statistics and enables planning across route boundaries.

- *Value Normalization*: Terminal rewards can vary significantly in scale across episodes. To stabilize learning, we normalize rewards online:

$$\tilde{z} = \text{clip}\left(\frac{z-\mu}{\sigma+\epsilon}, -3, 3\right),$$

  where running statistics (μ, σ) are updated with all raw rewards before normalizing.
- *Temperature Schedule*: The temperature τ in Eq. (7) follows a schedule based on training progress:

$$\tau(\texttt{progress}) = \begin{cases} 1.0 & \text{if } \texttt{progress} < 0.3, \\ 0.5 & \text{if } 0.3 \le \texttt{progress} < 0.6, \\ 0.1 & \text{otherwise.} \end{cases}$$

During evaluation, we use greedy selection ($\tau = 0$) and disable Dirichlet noise.

Algorithm 2 End-to-End RL for TRNDP

Input: Graph $G = (V, E)$, OD matrix D, directed edges $\mathcal{I}$, edge features Z, routes K, max length $L_{\max}$, update frequency M
Output: Policy π_θ (yields routes $\Pi = \{r_1, \ldots, r_K\}$)
Initialize: Buffer $B \leftarrow \varnothing$
while $t < t_{\max}$ **do**
  // Parallelized data collection
  for each worker $w \in \{1, \ldots, N\}$ **in parallel do**
    $\Pi \leftarrow \varnothing$; $V_{\text{cmp}} \leftarrow \varnothing$
    for $k = 1$ to K **do**
      $r_k \leftarrow [\,]$; $V_{\text{cur}} \leftarrow \varnothing$
      while $|r_k| < L_{\max}$ **do**
        $\mathcal{C}_t \leftarrow$ valid one-hop neighbors of frontier
        if $\mathcal{C}_t = \varnothing$ **then**
          break
        end if
        $s_t \leftarrow \textsc{FormState}(V_{\text{cur}}, V_{\text{cmp}}, \mathcal{I}, Z)$
        $m_t \leftarrow \textsc{Mask}(\mathcal{C}_t, n)$
        $a_t \sim \pi_\theta(\cdot \mid s_t)$ using m_t
        Append a_t to r_k; $V_{\text{cur}} \leftarrow$ nodes in r_k
        if $|r_k| < L_{\max}$ **then**
          $r_t \leftarrow \mathcal{R}_{\text{partial}}$ (Eq. 18)
        else
          $r_t \leftarrow \mathcal{R}$ (Eq. 3)
        end if
        Append (s_t, a_t, r_t, m_t) to local buffer
      end while
      $V_{\text{cmp}} \leftarrow V_{\text{cmp}} \cup V_{\text{cur}}$; $\Pi \leftarrow \Pi \cup \{r_k\}$
    end for
    // Compute GAE advantages $\hat{A}_t$; stream to B
  end for
  if $|B| \ge M$ **then**
    // Policy optimization
    for epoch $= 1$ to K_{epochs} **do**
      for minibatch $b \sim B$ **do**
        $\theta \leftarrow \theta - \eta \nabla_\theta (L_{\text{clip}} + c_v L_V - c_e H)$
      end for
    end for
    $B \leftarrow \varnothing$; broadcast θ to workers
  end if
end while
return π_θ

Simulation		**Policy Network**		**AlphaTransit**		**End-to-End RL (PPO)**	
Horizon	10,000	Node features	16	Env steps $t_{\max}$	10^6	Env steps	10^6
Time step	1s	Edge features	2	MCTS sims N_{iter}	400	Learning rate η	10^{-4}
Bus capacity	40	GATv2 blocks	4	PUCT c	1.5	Discount γ	0.99
Stop duration	60s	Activation	tanh	Dirichlet α_{dir}	0.3	GAE λ	0.95
Routes K	16	Channels	128, 128, 64, 64	Dirichlet ε	0.25	Clip ϵ	0.2
Max length $L_{\max}$	14	Attn heads	8, 8, 4, 4	Buffer size	100k	Epochs	4
Modal split α	0.3, 1.0	Actor MLP	256, 128, 64	Workers W	8	Batch size	32
Comfort thresh	1.0	Critic MLP	256, 128, 64	Batch size	256	Update freq	128
Stop spacing	1	Attention dropout	0, 0.05, 0.1, 0.1	Learning rate η	5×10^{-5}	Entropy coef	0.01
Platoon Δn	5	Hidden dim	64	Iterations	400	Value coef	0.5

Table 2. Hyperparameters for simulation, policy network, and training. Simulation parameters define the transit environment and are shared across all methods. The policy network architecture is identical for both AlphaTransit and End-to-End RL. Temperature τ is annealed from 1.0 to 0.1 over training.

H. Limitations and Future Work

There are several limitations and promising directions for future work. First, our current evaluation focuses on a single mid-sized city; extending the benchmark and training procedure to additional cities and larger metropolitan networks is essential for assessing generalization and scalability. That said, our dataset uniquely integrates real-world network topology, census-derived demand, and actual operating routes (Appendix B). This level of realism is absent in existing benchmarks, and curating such comprehensive datasets for additional cities is a non-trivial endeavor. Second, our simulator and demand model reflect a static peak-hour approximation; incorporating time-varying demand, stochastic disruptions, and reliability objectives would better match operational practice. Finally, transit planning often requires explicit equity, accessibility, and robustness constraints; incorporating these objectives into the reward and analysis framework is a natural extension.

I. Code and Data

Our code and dataset are publicly available at: `https://anonymous.4open.science/r/AlphaTransit`. The dataset includes three CSV files and one JSON file (nodes, edges, demand, and existing routes) to support future research in related areas.